ELTE IK - Computer Science BSc

Final Exam Topics

15. Data structures and data types

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Data structures and data types

Arrays, stacks, queues, linked lists; binary trees, general trees, traversals, representations; binary heaps, priority queues; binary search trees and their operations, AVL trees, B+ trees; hash tables, hash functions, key collisions and their resolutions: by chaining, open addressing, probe sequences; representations of graphs.

1 Simple data types

1.1 Data type

Data structure: $\sim$ structure.
Data type: a data structure together with its associated operations.
Data structures:

- *Array*: a sequence of elements of the same type, fixed size.
- *Stack*: We always put the next element on top of the stack, and we can only query and remove the topmost one.

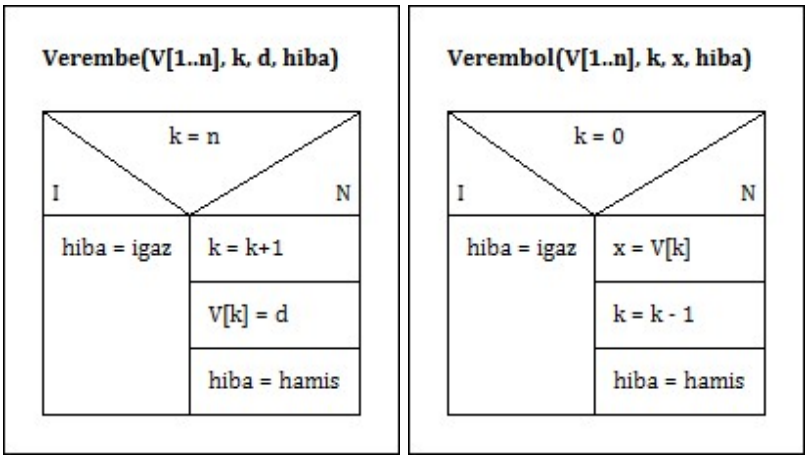


Figure 1: Stack operations

- *Queue*: Simple, priority, and double-ended. In a priority queue the elements have an associated value by which they can be ordered.

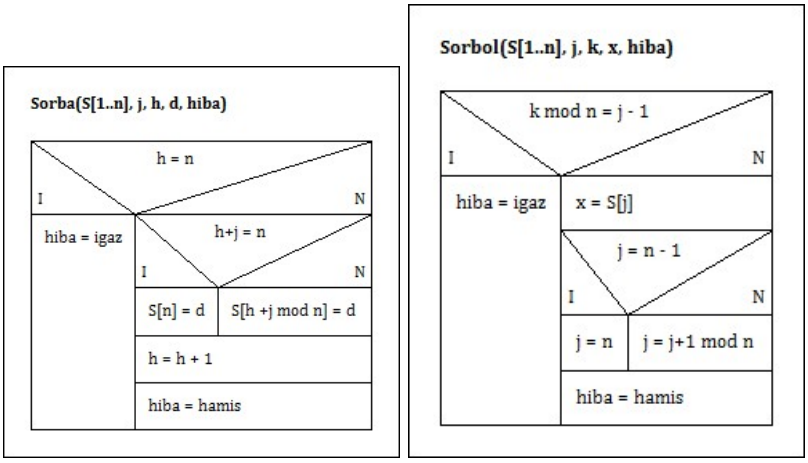


Figure 2: Queue operations

- *List*: Represented with a linked representation. We can distinguish lists by 3 criteria: with/without a head element, direction of chaining one/two, with/without cyclicity. If our list has a head element, then the head element exists even if the list is empty.
  The list consists of nodes, each node has a pointer pointing to the next one, and possibly to the previous one as well, if it is bidirectional. In addition there is also a pointer to a first and to a current node, and the pointer of the last element is NIL. We can implement the list so that an element can be inserted into, or deleted from, an arbitrary position.

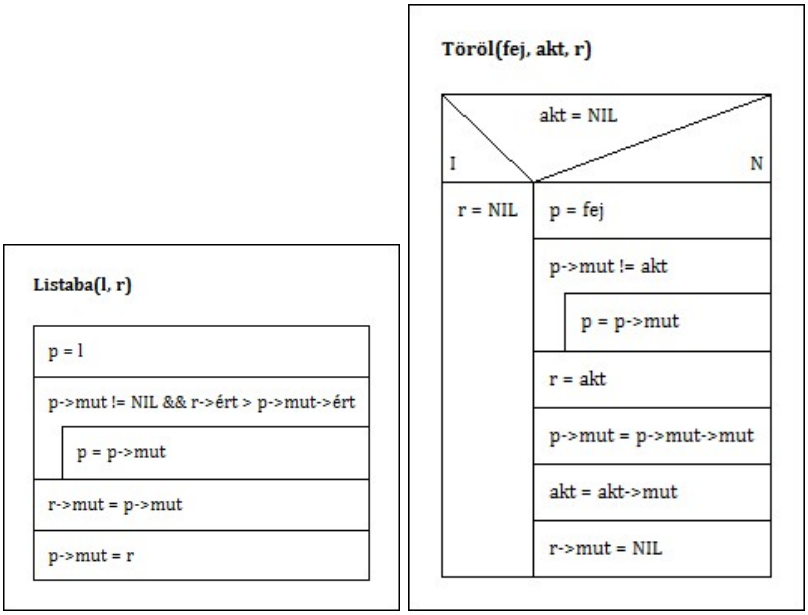


Figure 3: List operations

2 Trees and their traversals, representations

2.1 Binary tree

Trees are often used for representing large data sets and multisets, but also for other data-representation purposes. In the case of (binary) trees, every data element, or by its usual name node, has at most two successors: a left and/or a right successor.

Concepts:

- *child*: the left or right successor of the node

- *parent*: the node of the children
- *siblings*: children of the same node
- *leaf*: a parent without children
- *root node*: has no parent
- *internal node*: a non-leaf node
- *descendants*: the children of a node and their descendants
- *ancestors*: the parent of a node and its ancestors

2.2 General tree

The concept of a binary tree can be generalized. If in the tree an arbitrary node has at most r successors, we speak of an r-ary tree. In this case the children of a node and the subtrees belonging to them are usually numbered with selectors in [0..r). If a node has no i-th child (i ϵ [0..r)), then the i-th subtree is empty. So a binary tree and a 2-ary tree essentially mean the same thing, with the selector correspondence left $\sim$ 0 and right $\sim$ 1 applied here.

The levels of the tree are determined as follows. The root is at level zero. The children of the i-th level nodes are found at the (i + 1)-th level. The height of the tree equals the level number of its deepest-lying leaves. The height of the empty tree is h($\oslash$) = -1.

The trees discussed here are also called rooted trees, because they can be regarded as directed graphs whose edges are directed from the root node toward the leaves, and from the root every node can be reached by exactly one path.

2.3 Their traversals

Programs that work with trees are often related to some of the four classic traversals, which visit the nodes of the tree in a given order, and call the same operation on every node, regarding which we require that its running time be Θ (1) (which of course can still be a composite operation). Processing the *f node can be, for example, printing f key. For the empty tree every traversal is the empty program. *For non-empty r-ary trees*

- *Preorder*: first it processes the root of the tree, then it traverses the 0..r - 1-th subtrees in order;

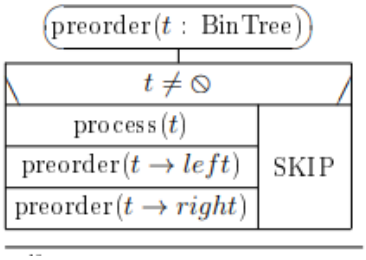


Figure 4: Preorder traversal

- *Inorder*: first it traverses the zeroth subtree, then it processes the root of the tree, then it traverses the 1..r - 1-th subtrees in order;

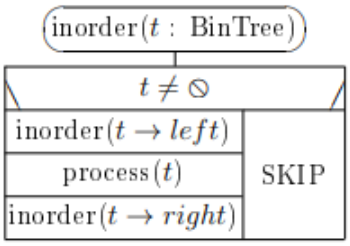


Figure 5: Inorder traversal

- *Postorder*: first it traverses the 0..r - 1-th subtrees in order, and it processes the root of the tree only after the subtrees

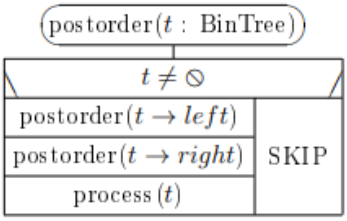


Figure 6: Postorder traversal

- *LevelOrder*: it processes the nodes level by level starting from the root, traversing each level from left to right.

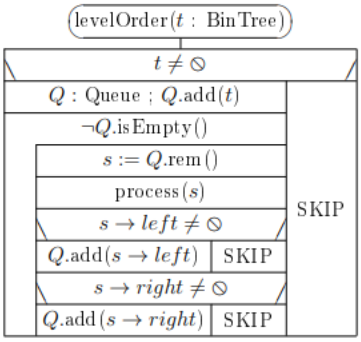


Figure 7: LevelOrder traversal

So the first three traversals are very similar to each other. Their names tell us when they process the root node relative to the subtrees.

2.4 Their representations

2.4.1 Graphical

The most natural and one of the most frequently used is the linked representation of binary trees:

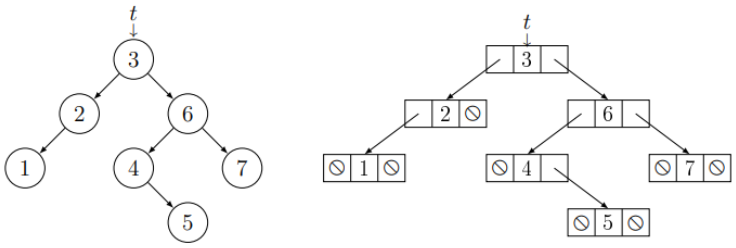


Figure 8: The same binary tree with graphical and linked representation.

2.4.2 Abstract

The representation of the empty tree is the ⊘ pointer, so its notation is the same as for abstract trees. The nodes of the binary tree can be represented, e.g., as objects of the class below, where the representation of the BinTree abstract type is simply Node*.

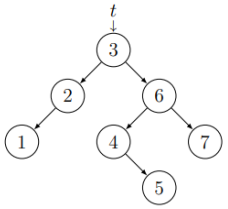
Node
+ key : $\mathcal{T}$ // $\mathcal{T}$ valamilyen ismert típus + $left, right$: Node*
+ Node() { $left := right := \oslash$ } // egycsúcsú fát képez belőle + Node(x : $\mathcal{T}$) { $left := right := \oslash$; $key := x$ }

Sometimes it is useful if the nodes also have a parent pointer, because in the tree this way we can also move upward.

Node3
+ key : $\mathcal{T}$ // $\mathcal{T}$ valamilyen ismert típus + $left, right, parent$: Node3*
+ Node3(p:Node3*) { $left := right := \oslash$; $parent := p$ } + Node3(x : $\mathcal{T}$, p:Node3*) { $left := right := \oslash$; $parent := p$; $key := x$ }

2.4.3 Parenthesized

The parenthesized, i.e. textual, form of an arbitrary non-empty binary tree: (leftSubtree Root rightSubtree) The empty tree is represented by the empty string. For easier readability we can use several kinds of parenthesis pairs. The lexical elements of the parenthesized representation are: opening parenthesis, closing parenthesis, and the labels of the nodes.



A balra látható t bináris fa

- egyszerű zárójeles alakja:
  (((1) 2) 3 ((4 (5)) 6 (7)))
- elegáns zárójeles alakja:
  { [(1) 2] 3 [(4 ⟨5⟩) 6 (7)] }

Figure 9: The same binary tree with graphical and textual representation.

3 Heap and queue

The binary heap is a concrete data structure that is used to implement the priority queue. The priority queue is a more general concept that can encompass several kinds of data structures or algorithms from the point of view of ordering by priority and accessing the elements.

3.1 Binary heap

A complete binary tree, in which the priority of the elements is determined based on the keys found in the nodes. The binary heap efficiently supports the priority queue operations, for example inserting an element and removing the element with the highest priority.

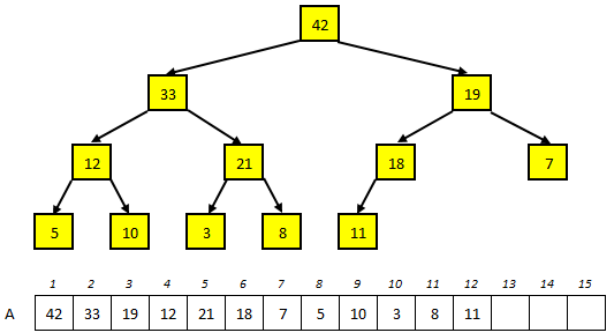


Figure 10: Example

3.2 Its methods

3.2.1 Adding

We add a new element to the heap at the leaf level (at the nearest free position). Then, moving upward in the heap, we compare the new element with its parent. If the priority of the new element is greater, then we swap the elements, and continue moving upward to the level one higher. We repeat this until the priority of the new element is appropriate for the properties of the binary heap.

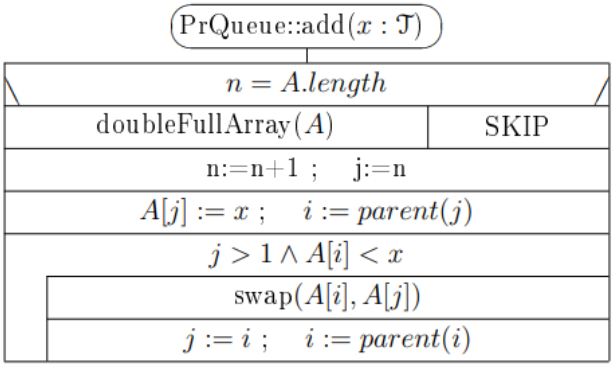


Figure 11: The algorithm of add(x)

3.2.2 Deletion

The element with the highest priority is always found at the root of the heap. We remove (remMax) this element, which is the topmost element in the heap. At this point the heap will be empty, or the root element has to be replaced with the last element found at the bottom. If the heap is not empty, we move the replacement element downward in the heap to restore the heap properties. We compare the element with its child, then swap it with the child of smaller priority (sink), if that is greater. We repeat this process until the element gets to the appropriate place in the heap.

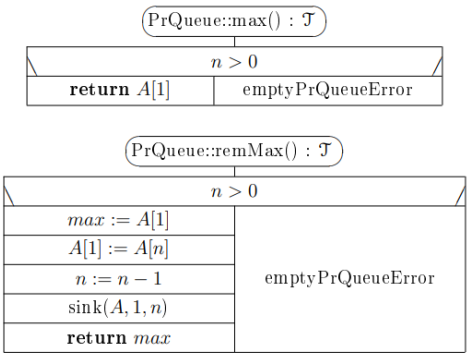


Figure 12: The algorithm of remMax()

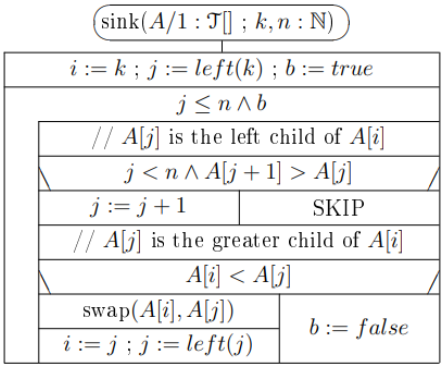


Figure 13: The algorithm of sink(x)

4 Special trees

4.1 Binary search tree

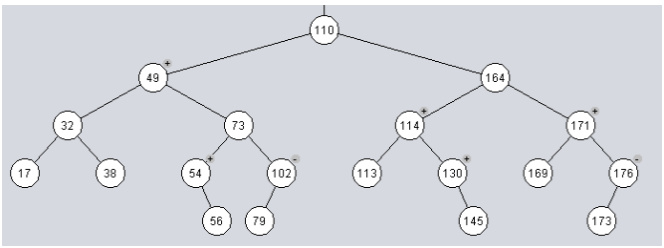


Figure 14: Example of a binary search tree

A binary search tree is a data structure used for the efficient storage and querying of ordered data. *Its properties*

- *Orderedness*: Every node has a key, based on which it can be decided whether it is located in the left subtree (smaller than the node) or in the right subtree (greater than the node).
- *Fast search*: It enables fast search thanks to the orderedness. So, if we search for an element and start from the node, we immediately know whether the searched element is to the right or to the left.

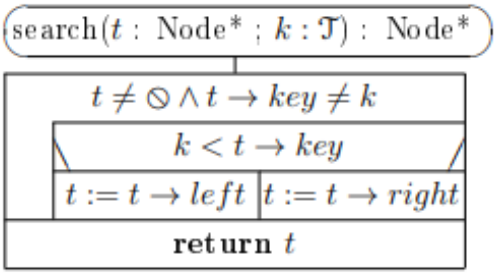


Figure 15: Search in a binary search tree

- *Insertion and deletion*: At insertion we fit the new element into the appropriate place, taking the orderedness into account. At deletion the structure has to be changed so that the orderedness is preserved.

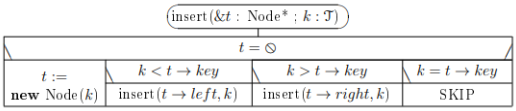


Figure 16: Insertion into a binary search tree

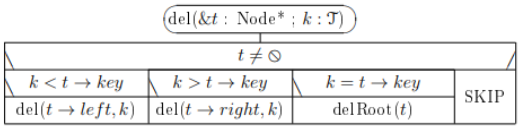


Figure 17: Deletion from a binary search tree

- *In-order traversal*: Using this traversal, we obtain the elements of the binary search tree in sorted order.

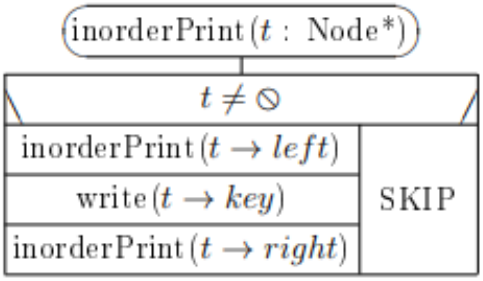


Figure 18: In-order traversal in a binary search tree

4.2 AVL tree

The AVL tree is a special binary search tree. Its goal is to maintain balance at every node of the tree, so that the search, insertion and deletion operations can be performed efficiently. So in addition to the properties of the binary search tree it has two more: height and self-balancing.

4.2.1 Height property

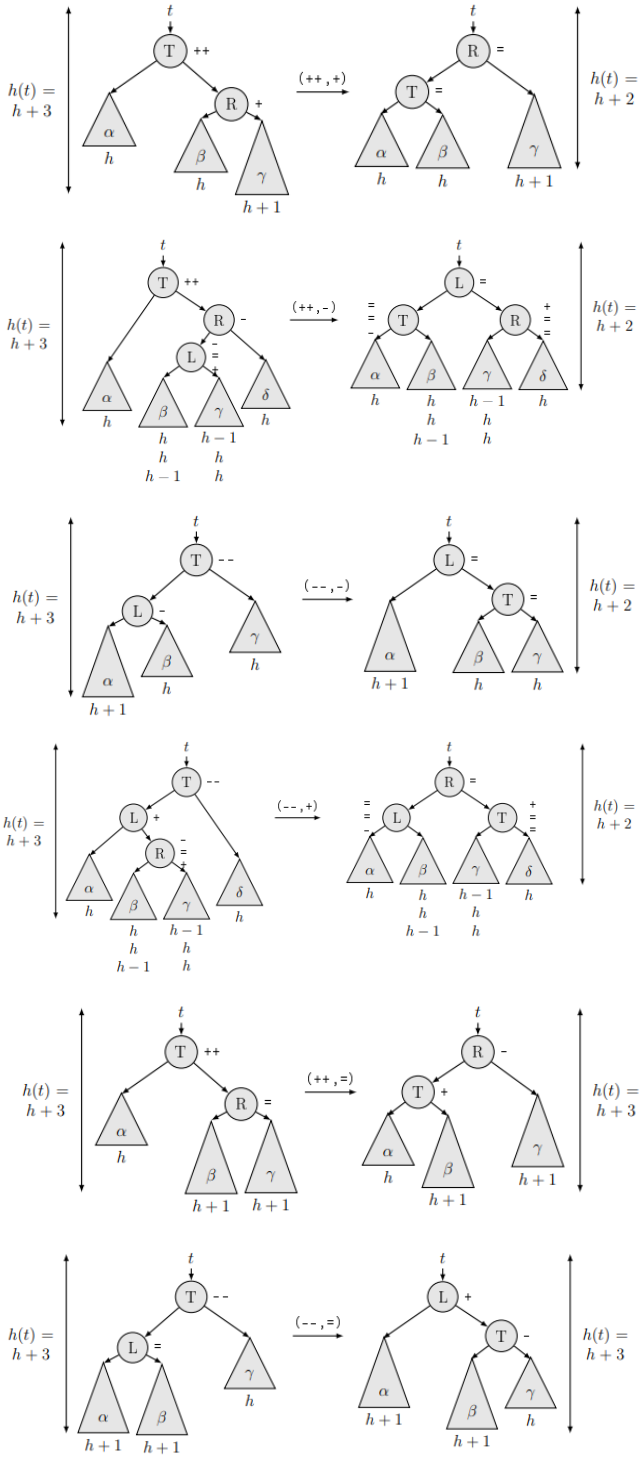
The AVL tree uses the height property in order to maintain balance. Every node has a height value, which is the height of the farthest leaf from the node. In an AVL tree the height difference of all nodes can be at most 1, that is, the AVL tree is in balance.

4.2.2 Self-balancing

If after an insertion or deletion operation the balance of the AVL tree is violated, then rotation operations serving the rebalancing of the tree are performed. The goal of the rotations is to correct the height difference and restore the balance of the AVL tree.

4.2.3 Rotations

Depending on which side changed and by how much, the following rotations have to be used. If the left subtree sinks one level then it gets a $-$ marking, if the right one then a $+$ marking. The $++/--$ means that on one side the height of the subtree is greater by two than on the other.



4.3 B+ tree

The B+ tree, in which every node contains at most d pointers ($4 \leq d$), and at most d-1 keys, where d is a constant characteristic of the tree, the degree of the B+ tree. We regard each reference in the internal nodes as being "between" two keys, that is, it points to the root of a subtree in which every value is found between the two keys.

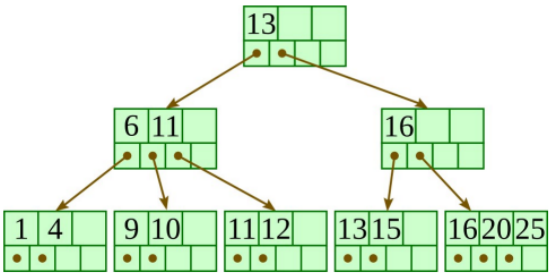


Figure 19: B+ tree of degree 4

An arbitrary B+ tree of degree d satisfies the following invariants, where $4 \leq d$ is a constant:

- In every leaf there are at most d-1 keys, and the same number of pointers referencing the corresponding data record.
- Every leaf is the same distance from the root.
- In every internal node there is one more pointer than keys, where d is the upper bound for the number of pointers.
- For every internal node N, where k is the number of keys in node N: in the subtree belonging to the first child every key is smaller than the first key of N; in the subtree belonging to the last child every key is greater than or equal to the last key of N; and for any key r in the subtree belonging to the i-th child ($2 \leq i \leq k$), N.key[i-1] $\leq$ r $<$ N.key[i].
- The root node has at least two children. Except if it is the only node of the tree.
- Every internal node different from the root has at least floor(d/2) children.
- Every leaf contains at least floor(d/2) keys. Every key of the data set represented by the B+ tree appears in some leaf, in strictly monotonically increasing order from left to right.

4.3.1 Insertion

If the tree is empty, create a new leaf node whose content is the key/pointer pair to be inserted! Otherwise find the leaf corresponding to the key! If the key already appears in the leaf, the insertion fails. Otherwise:

- If there is free space in the node, insert the appropriate key/pointer pair, ordered by key, into this node!
- If the node is already full, split it into two nodes, in the middle, and distribute the d keys equally between the two nodes! If the resulting node is a leaf, take a copy of the smallest value of the second created node, and repeat this insertion algorithm to insert it into the parent node! If the node is not a leaf, take out the middle value during the distribution of the keys, and repeat this insertion algorithm to insert this middle value into the parent node! (If needed, the parent node is created first. At this point the height of the B+ tree increases.)

4.3.2 Deletion

Find the leaf containing the key to be deleted! If there is no such leaf, the deletion fails.

- If the leaf node found during the search is also the root node:

  - Delete the appropriate key and its associated pointer from the node!
  - If the node still contains a key, we are done.
  - Otherwise delete the single node of the tree, and we get an empty tree.

- The leaf node found during the search is not the root node:
  - Delete the appropriate key and its associated pointer from the leaf node!
  - If the leaf node still contains enough keys and pointers to satisfy the invariants, we are done.
  - If the leaf node now has too few keys to satisfy the invariants, but its next or its preceding sibling has more than necessary, distribute the keys equally between it and the appropriate sibling! In the common parent of the two siblings, rewrite the splitting key belonging to the two siblings to the minimum of the second of the two siblings!
  - If the leaf node now has too few keys to satisfy the invariant, and both its next and its preceding sibling are at the minimum needed to satisfy the invariant, then merge it with a neighboring sibling! In doing so, the keys and their associated pointers from the second (in left-to-right order) of the two siblings are copied in sequence into the first, after its original keys and pointers, and then the second sibling is deleted. After this we have to repeat the deletion algorithm on the parent, to remove from the parent the splitting key (which until now separated the now-merged leaf nodes), together with the pointer referencing the now-deleted second sibling.

- Deletion from an internal node – different from the root:
  - Delete from the internal node the splitting key between the just-merged two children of the internal node and the pointer referencing the child deleted during the merge!
  - If the internal node still has floor(d/2) children (to satisfy the invariants), we are done.
  - If the internal node now has too few children to satisfy the invariants, but its next or its preceding sibling has more than necessary, distribute the children and the splitting keys between them equally between it and the appropriate sibling, including among the splitting keys the splitting key between the siblings (in their common parent)! During the redistribution of the children and the splitting keys, the middle splitting key takes the place of the old splitting key belonging to the two siblings in the siblings' common parent, so that it appropriately represents the split point that has changed between them! (If the total number of children in the two siblings is odd, then after the redistribution as well, let the sibling who had more before also have more children!)
  - If the internal node now has too few children to satisfy the invariant, and both its next and its preceding sibling are at the minimum to satisfy the invariant, then merge it with a neighboring sibling! Create the merged node from the first (in left-to-right order) of the two siblings. Its children and splitting keys are first its own children and splitting keys in the original order, followed by the splitting key between the two siblings (in their common parent), and finally come the children and splitting keys of the second sibling, also in the original order. After this delete the second sibling. After merging the two siblings we have to repeat the deletion algorithm on their common parent, to remove from the parent the splitting key (which until now separated the now-merged siblings), together with the pointer referencing the now-deleted second sibling.

- Deletion from the root node, if it is not a leaf node:
  - Delete from the root node the splitting key between the just-merged two children of the root node and the pointer referencing the child deleted during the merge!
  - If the root node still has 2 children, we are done.
  - If the root node has only 1 child remaining, then delete the root node, and let its single remaining child be the new root node! (At this point the height of the B+ tree decreases.)

5 Hash tables

The hash table, also called hash table or hash map, is an efficient data structure that stores key-value pairs. The hash table uses keys for fast access to the stored values. A hash table is essentially an array that stores and searches the data with the help of a so-called hash function.

5.1 Hash functions

When we want to add a value belonging to a key to the hash table, we first apply a hash function to the key. The hash function is an algorithm that generates a unique hash code based on the key. The hash code determines an index in the array where we will store the value.

- *Storage:* Based on the hash code generated in the previous step, we place the value at the appropriate index of the array in the hash table. If two or more keys happen to have the same hash code, then we call this a hash collision.
- *Handling hash collisions:* Handling hash collisions plays a critical role in the efficiency of the hash table.
  - *Chaining:* Each hash code has a "chain" that stores the colliding keys in a data structure (usually a linked list or array). If a new key has the same hash code, it goes onto the corresponding chain, and the value is added to the linked list or array.
  - *Open addressing:* The colliding keys are stored directly in the array, without using a separate data structure. If a new key's hash code collides, we try various empty places in the array until we find an empty place for storage.
- *Search:* During the search operation we again apply the hash function to the searched key, and determine the place where the value is stored in the hash table. If we use chaining, then we walk through the linked list to search in the hash table for the value belonging to the key; we apply the hash function to the searched key, and determine the place where the value is stored in the hash table. If we use chaining, then we walk through the linked list or array, and compare the keys to find the appropriate value.
- *Deletion:* The deletion operation is similar to search. First we apply the hash function to the key, and determine the place where the value is stored in the hash table. If we use chaining, then we walk through the linked list or array, find the appropriate value, and delete it.

Among the advantages of the hash table is fast data access. If the hash function is well designed and we handle the hash collisions efficiently, the average time complexity of search, insertion and deletion operations can approach O(1), which is very efficient. However, the hash table also raises some limitations. First, the hash function does not always guarantee completely unique hash codes, so there is the possibility of hash collisions occurring. This has to be handled with appropriate collision handling. Second, the hash table consumes memory, especially if it has to contain a large array. Finding the balance between the amount of data and the efficiency of the hash function is an important consideration for efficiency.

5.2 Probe sequence

The probe sequence is a method applied in data structures such as hash tables or buckets, used to determine the unique place of keys. When we insert or search for a key in a hash table or buckets, we use the probe sequence to determine where the key's storage place is found in the table. The probe sequence is a sequence or order generated based on the hash code or the key. Among the most common probe sequences are:

- *Linear probing:* The chosen hash code or index is already occupied in the table, so the algorithm tries successive consecutive indices until it finds an empty place. For example, if the original hash code is 5, and index 5 is already occupied, the algorithm tries index 6, then 7, and so on.

- *Quadratic probing:* The algorithm increases the index in a quadratic manner in case of collision. For example, if the original hash code is 5, and index 5 is already occupied, the algorithm tries index $(5 + 1^2 = 6)$, then $(5 + 2^2 = 9)$, and so on.
- *Double hashing probe sequence:* The algorithm uses a second hash function in case of collision to determine the next index. The second hash function generates a different hash code, which helps avoid clustering and cycles among the indices.

6 Representations of graphs

- *Adjacency matrix:* The adjacency matrix is an n x n matrix, where n is the number of vertices of the graph. The element in row i and column j indicates the presence or weight of the edges between vertices i and j. If the edges are directed, then the matrix is not symmetric. This representation stores the structure of the graph efficiently, but its resource requirement grows in quadratic proportion to the number of vertices.
- *Edge list:* The edge list is a list that contains the data of all edges. Every edge has a start vertex, an end vertex, and possible further properties, such as weight. This representation requires little memory, but querying the structure of the graph can be more time-consuming.
- *Adjacency list:* The adjacency list contains a list belonging to every vertex, in which the vertices directly adjacent to the vertex, or the edges, are listed. This representation is generally more efficient for sparse graphs, since it only stores the actually adjacent vertices. However, querying the structure of the graph can be of linear time requirement.
- *Incidence matrix:* The incidence matrix is an n x m matrix, where n is the number of vertices, and m is the number of edges. The element in row i and column j is 1 if vertex i is involved in edge j, otherwise it can be 0 or another marking. This representation stores the structure of the graph and the attributes of the edges efficiently, but its resource requirement is proportional to the number of vertices and edges.